le cnam

Conservatoire national des arts et métiers

École doctorale Sciences des Métiers de l'Ingénieur

Laboratoire de Mécanique des Structures et des Systèmes Couplés

Thèse de doctorat

Présentée par : **Mathieu Aucejo**

Soutenue le : **15 juin 2024**

Discipline : **Sciences de l'ingénieur**

Spécialité : **Mécanique**

Cnam thesis template
User's guide

Thèse dirigée par :

Henri Grégoire, Abbé constitutionnelle, Cnam, Paris

Henri Tresca, Professeur titulaire de la Chaire de Mécanique, Cnam, Paris

Et co-dirigée par :

Pierre-Simon Laplace, Professeur de tout, Institut de France, Paris

Joseph Fourier, Membre de l'Académie des Sciences, Académie des sciences, Paris

Jury

Jean-Antoine Chaptal	Professeur des Unversités, Chaire de Chimie Appliquée, Conservatoire national des arts et métiers, Paris, France	Président du jury
Donald Ervin Knuth	Professeur émérite, Département d'Informatique, Université de Stanford, Palo, Alto, Californie, États-Unis	Rapporteur
Ada Lovelace	Maître de conférences HDR, Département de Mathématiques, Université de Londres, Royaume-Uni	Rapportrice
Jacques de Vaucanson	Inspecteur général des manufactures, Académie royale des sciences, Paris, France	Examinateur
Henri Grégoire	Abbé constitutionnelle, Conservatoire national des arts et métiers, Paris, France	Directeur de thèse
Henri Tresca	Professeur titulaire de la Chaire de Mécanique, Conservatoire national des arts et métiers, Paris, France	Co-directeur de thèse

Software and cathedrals are almost the same thing: first we build them, then we pray.
Sam Redwine

Foreword

This document is the user guide for the Typst template dedicated to theses at the Conservatoire national des arts et métiers (Cnam). It was designed to help doctoral and master's students write their thesis with Typst, a modern and efficient alternative to LaTeX.

This guide uses the full set of features provided by the template and includes practical examples. It aims to help doctoral and master's students write their thesis efficiently and professionally while following Cnam standards and requirements.

I also thank my colleague Éric Bavu, who motivated me to create this template after publishing a Quarto + LaTeX template for Cnam theses. The Typst template is inspired by his work. Thank you, Éric!

Finally, I thank the doctoral and master's students who already use, or will use, this template for their feedback and suggestions, which will help continuously improve this tool.

Mathieu Aucejo
June 2026

Résumé

Ce travail présente et documente le template Typst `community-cnam-thesis`, conçu pour permettre aux doctorants du Conservatoire national des arts et métiers (Cnam) de rédiger leur thèse dans un environnement moderne, lisible et reproductible. Le template vise la production d'un PDF conforme à la maquette institutionnelle 2024-2025, avec une attention particulière à la qualité typographique, à la structuration du manuscrit et à la compatibilité avec les exigences d'archivage (notamment PDF/A). Le guide se compose de six chapitres, chacun dédié à une grande famille de fonctionnalités : structure, figures et tableaux, équations, boîtes informationnelles, ainsi que références et annotations. La plupart des contenus suivent le principe « code + rendu » : un extrait Typst présente la syntaxe, puis son résultat est affiché immédiatement dans le document. Cette logique permet de considérer non seulement comme un guide pratique d'utilisation, mais également comme une démonstration des possibilités offertes par le template.

Mots-clés : Typst, template, thèse, Cnam, PDF/A, publication scientifique reproductible, guide d'utilisation, documentation.

Abstract

This work presents and documents the Typst template `community-cnam-thesis`, designed to enable doctoral candidates at the Conservatoire national des arts et métiers (Cnam) to write their thesis in a modern, readable, and reproducible environment. The template aims to produce a PDF compliant with the 2024-2025 institutional format, with particular attention to typographic quality, manuscript structure, and compatibility with archiving requirements (notably PDF/A). The guide consists of six chapters, each devoted to a major feature category: structure, figures and tables, equations, informational boxes, as well as references and annotations. Most of the content follows the "code + output" principle: a Typst excerpt presents the syntax, and its result is displayed immediately in the document. This approach allows it to be considered not only as a practical user guide, but also as a demonstration of the possibilities offered by the template.

Keywords: Typst, template, thesis, Cnam, PDF/A, reproducible scientific publishing, user guide, documentation.

Table of contents

Foreword i

Résumé iii

Abstract v

User's guide 11

Introduction . . . 13
Context and goals . . . 13
Guide structure . . . 13
About Typst . . . 13
Installing and using Typst CLI . . . 14
Tooling . . . 15
Editors and IDEs . . . 15
Package management . . . 15

1. General Usage . . . 17
1.1. General information . . . 18
1.1.1. Fonts . . . 18
1.1.2. Theme colors . . . 18
1.2. Template initialization . . . 18
1.3. File tree . . . 22
1.4. Structure of the main file . . . 22

2. Document Formatting . . . 25
2.1. Title Hierarchy . . . 26
2.1.1. Numbered and Unnumbered Chapters . . . 26
2.1.2. Numbered and Unnumbered Sections . . . 26
2.2. Cross-References . . . 27
2.3. Text Formatting . . . 27
2.3.1. Emphasis and Code . . . 27
2.3.2. Hyperlinks . . . 27
2.3.3. Lists . . . 27
2.3.4. Quotes . . . 28
2.3.5. Footnotes . . . 28

2.3.6. Page Breaks . . . 28
2.4. Parts and Page Layout Environments . . . 29
2.5. Back Cover . . . 29

3. Figures and tables . . . 31
3.1. Figures . . . 32
3.2. Tables . . . 33
3.3. Long and short titles . . . 35
3.4. Creating plots directly in Typst . . . 35

4. Equations and algorithms . . . 39
4.1. Inline equations . . . 40
4.2. Numbered and unnumbered equations . . . 40
4.3. Advanced environments . . . 41
4.4. Algorithms . . . 42

5. Callout boxes . . . 43
5.1. Available boxes . . . 44
5.2. Special cases for code boxes . . . 45

6. References . . . 47
6.1. Bibliographic references . . . 48
6.1.1. File formats . . . 48
6.1.2. Basic syntax . . . 48
6.1.3. Multiple references . . . 48
6.2. Glossary, acronyms, and nomenclatures . . . 49

7. Review comments . . . 51
7.1. Collaborative comments . . . 52
7.2. Comment types . . . 52
7.3. Annotation syntax . . . 53
7.4. Table of annotations . . . 55

Conclusion . . . 57
Feature summary . . . 57

Bibliography . . . 59

Appendices **61**

A. PDF/A-1b validation . . . 63
A.1. Submission on theses.fr . . . 64
A.2. CINES validation . . . 64

A.3. Typst compilation to PDF/A-1b 64

List of figures

Figure 3.1. Official Cnam logo 32

Figure 3.2. (a) Cnam logo and (b) Victory stamp 33

Figure 3.3. Representation of a rectangle 35

Figure 3.4. Free response of a one-degree-of-freedom mechanical system 37

Figure A.1. Selecting the PDF/A-1b format in the Typst web app 64

Figure A.2. Selecting the PDF/A-1b format in the Tinymist extension for Visual Studio Code 65

List of tables

Table 3.1. Table of the perimeters of a few geometric shapes 34

Review comments

1 Abbé Grégoire : This is a blue comment. 54

2 Henri Tresca : I think this is great! 54

3 Abbé Grégoire : I am not sure I understand this remark, Henri? 54

4 Abbé Grégoire : Since I don't have any ideas for writing a long and funny text, I will use a classic `#lorem()`. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat. 54

5 Mathieu Aucejo : I wonder whether this is really necessary? 55

Summary of review comments

Note	2
Comment	1
Question	2
Todo	0
Total	5

User's guide

Introduction

Context and goals

Doctoral candidates at Cnam must write their thesis according to specific requirements: page layout, fonts, colors, and document structure must follow institutional rules. However, not everyone wants to use LaTeX, which is often perceived as complex. Since 2023, Typst has emerged as a modern and efficient alternative. Typst is a markup language for producing high-quality documents with concise syntax, and it is especially suited to thesis writing thanks to native support for references, figures, tables, and equations.

The purpose of this guide is to present the Typst template for Cnam theses and provide practical usage instructions. It is not a full introduction to Typst itself, but a focused guide for thesis writing with this template.

For more information about Typst, see the official Typst website and the book Typst Handbook: From Zero to Hero by Stefano Coelati Rama.

Guide structure

This guide is organized into seven thematic chapters:

1. **General usage** – General instructions for using the `community-cnam-thesis` template.
2. **Document formatting** – Heading hierarchy, numbering, cross-references, lists, etc.
3. **Figures and tables** – Image insertion, captions, table formatting, subfigures, and more.
4. **Equations and algorithms** – Equation syntax and algorithm insertion.
5. **Information boxes** – Box types, styles, colors, icons, etc.
6. **References and glossary** – Bibliography, file formats, glossary tools, etc.
7. **Review annotations** – Margin notes, comments, collaborative review.

About Typst

Typst is an open-source markup language written in Rust and developed from 2019 by Laurenz Mädje and Martin Haug. Version 0.1.0 was released on GitHub on April 4, 2023[1].

Typst positions itself as a modern, fast, and easier successor to LaTeX. Key advantages include:

- incremental compilation;
- clear error messages;
- a Turing-complete language;
- a coherent styling system for fonts, layout, margins, …

[1]GitHub repository: https://github.com/typst/typst

- an active community;
- a straightforward package ecosystem via Typst Universe
- editor extensions such as `Tinymist` for VS Code.

Installing and using Typst CLI

Typst CLI[2] is available on Windows, macOS, and Linux.

- Build from source:

Code

```
cargo install --locked typst-cli
```

- Windows:

Code

```
choco install typst
scoop install main/typst
winget install --id Typst.Typst
```

- Linux:

Code

```
sudo snap install typst
sudo dnf copr enable claaj/typst & sudo dnf install typst
sudo pacman -S typst
```

- macOS:

Code

```
brew install typst
sudo port install typst
```

Once installed, compile Typst files with:

Code

```
typst compile /path/to/file.typ
typst compile /path/to/file.typ /path/to/output.pdf
```

[2]Command-line utility

You can also enable live recompilation:

Code

```
typst watch file.typ
```

Tooling

Typst can be used with any text editor, but IDE tooling improves productivity with syntax highlighting, completion, real-time compilation, and PDF preview.

Editors and IDEs

- **Text editors**: any editor can work, depending on your preferences.
- **Visual Studio Code and derivatives**: install `Tinymist` for Typst language support, live preview, and diagnostics.

Note

`Tinymist` includes a Typst compiler, so Typst CLI is optional.

- **JetBrains IDEs**: install `Typst Support` (based on `Tinymist`) from the plugin marketplace.
- **Cloud IDE**: Typst has an official cloud IDE, with alternatives such as TypeTeX.

Package management

Unlike LaTeX distributions (TeXLive, MiKTeX), Typst packages are centralized in the Typst packages repository, discoverable through Typst Universe.

Import packages with:

Code

```
#import "@preview/package-name:version": *
#import "@preview/package-name:version": package-name
#import "@preview/package-name:version": func1, func2
```

For `community-cnam-thesis`:

Code

```
#import "@local/community-cnam-thesis:0.1.4": *
#import "@preview/community-cnam-thesis:0.1.0": ct
#import "@preview/community-cnam-thesis:0.1.0": info-box, warning-box
```

The first import downloads and caches the package automatically.

Community tools that help manage dependencies include:

- UTPM,
- GoTPM,
- Tyler,
- Typship.

Chapter

1

General Usage

This chapter presents the general instructions for using the `community-cnam-thesis` template. It is recommended to follow these instructions before starting to write your manuscript.

Content

1.1. General information 18
1.2. Template initialization 18
1.3. File tree 22
1.4. Structure of the main file 22

1.1. General information

The `community-cnam-thesis` template is based on `bookly`, a Typst package developed by the author of this document. It provides a consistent document structure, predefined layout styles, and features specific to academic documents.

This template customizes the `bookly` template to meet the specific requirements of theses from the Conservatoire national des arts et métiers (Cnam). It also includes additional features that are described in the following chapters.

1.1.1. Fonts

To use the `community-cnam-thesis` template, you must install the following fonts on your system:

- Text: `TeXGyrePagellaX` (download link), `Libertinus Serif` (download link) and `New Computer Modern` (included with Typst).
- Mathematics: `TeX Gyre Pagella Math` (download link), `Libertinus Math` (download link) and `New Computer Modern Math` (included with Typst).
- Code: `Cascadia Code` (download link), `Hack` (download link) and `DejaVu Sans Mono` (included with Typst).

1.1.2. Theme colors

The `community-cnam-thesis` template defines two main colors to ensure visual consistency throughout the document:

- Primary color:
- Secondary color:

Note

Colors are defined in the `dictionary` `cnam-colors`. You can access them using the standard Typst syntax: `cnam-colors.primary` and `cnam-colors.secondary`.

1.2. Template initialization

To use the template, you must import it into your main `typ` file. Assuming the template and the main file are in the same folder, simply insert the following command on the first line of the main file.

Code

```
#import "@local/community-cnam-thesis:0.1.4": *
```

Note

If you split your document into multiple files, you must insert the previous command in the preamble of each file.

After importing the template, it must be initialized by applying a display rule (`show` rule) with the `#community-cnam-thesis()` command and passing the required options with the `with` instruction in your main `typ` file:

Code

```
#show: community-cnam-thesis.with(
    title: "thesis title",
    author: "Author name",
    lang: "fr",
    open-right: true,
    thesis-info: thesis-info-default,
)
```

This initialization function contains a number of arguments detailed below. You can modify these arguments according to your needs.

Argument

`⟨title⟩: "Thesis title"` `string` | `content`

Thesis title

Argument

`⟨author⟩: "Author name"` `string` | `content`

Thesis author

Argument

`⟨lang⟩: "fr"` `string`

Document language.

Note

All languages supported by **bookly** are also supported by the **community-cnam-thesis** template.

Argument

`⟨open-right⟩: true` `bool`

If `true`, chapters start on a right-hand page. If `false`, chapters start on the next page.

Argument

`(thesis-info)`: `thesis-info-default` `dictionary`

Dictionary containing thesis-related information.

- `doctoral-school` `string` | `content` : Affiliated doctoral school
- `supervisor` `array` : Definition of the thesis supervisor(s). Each entry is defined by a `dictionary` containing the following keys:
  - `name` `string` | `content` : Supervisor name
  - `title` `string` | `content` : Supervisor title/status (MCF HDR, PU, PRCM, …)
  - `institution` `string` | `content` : Affiliated institution/company of the supervisor,
- `co-supervisor` `array` : Definition of the thesis co-supervisor(s). Each entry is defined by a `dictionary` containing the following keys:
  - `name` `string` | `content` : Co-supervisor name
  - `title` `string` | `content` : Co-supervisor title/status (MCF, MCF HDR, PU, PRCM, …)
  - `institution` `string` | `content` : Affiliated institution/company of the co-supervisor,
- `laboratory` `string` | `content` : Name of the affiliated laboratory
- `defense-date` `string` | `content` : Thesis defense date
- `discipline` `string` | `content` : Thesis discipline
- `speciality` `string` | `content` : Thesis specialty
- `committee` `array` : Members of the defense committee. Each entry is defined by a `dictionary` containing the following keys:
  - `name` `string` | `content` : Committee member name
  - `position` `string` | `content` : Committee member position
  - `role` `string` | `content` : Committee member role (President, Reviewer, Examiner, etc.)
- `dedication` `string` | `content` : Thesis dedication
- `logo` `array` : Path to the institution logo

Default values

- `doctoral-school`: "Sciences des Métiers de l'Ingénieur",
- supervisor: `(:)`,
- co-supervisor: `(:)`,
- laboratory: `none`,
- defense-date: `none`,
- discipline: "Sciences de l'ingénieur",
- speciality: "Mécanique",
- committee: `(:)`,
- dedication: `none`,

- logo: `none`

To define the `supervisor`, `co-supervisor`, and `committee` dictionaries, several approaches are possible:

1. Direct definition in Typst.

Code

```
// main.typ
#let supervisor = (
  (name: "Henri Grégoire", title: "Abbé constitutionnelle", institution: "Cnam,
Paris"),
  (name: "Henri Tresca", title: "Professeur titulaire de la Chaire de Mécanique",
institution: "Cnam, Paris"),
)

#show: community-cnam-thesis.with(
    thesis-info: (
        supervisor: supervisor,
    ),
)
```

2. Definition in a `json` file.

Code

```
// thesis-info.json
{
  "supervisor": [
    {
      "name": "Henri Grégoire",
      "title": "Abbé constitutionnelle",
      "institution": "Cnam, Paris"
    },
    {
      "name": "Henri Tresca",
      "title": "Professeur titulaire de la Chaire de Mécanique",
      "institution": "Cnam, Paris"
    }
  ]
}

// main.typ
#show: community-cnam-thesis.with(
    thesis-info: json("/path/to/thesis-info.json"),
)
```

3. Definition in a separate `yaml` file.

Code

```yaml
# thesis-info.yaml
supervisor:
  - name: "Henri Grégoire"
    title: "Abbé constitutionnelle"
    institution: "Cnam, Paris"
  - name: "Henri Tresca"
    title: "Professeur titulaire de la Chaire de Mécanique"
    institution: "Cnam, Paris"
```

```typst
// main.typ
#show: community-cnam-thesis.with(
    thesis-info: yaml("/path/to/thesis-info.yaml"),
)
```

1.3. File tree

When writing a long document, such as a thesis manuscript, it is preferable to adopt a multi-file organization to avoid having one veeeeeery long main file. This is especially important during proofreading/correction, as well as in a collaborative writing context. I generally use the following structure:

```
main.typ
├─ appendices/
│  └─ app1.typ
├─ bibliography/
│  └─ biblio.bib
├─ chapters/
│  └─ chapter1.typ
├─ images/
│  └─ logo.png
└─ preamble/
   └─ summary.typ
```

1.4. Structure of the main file

Based on the structure defined in the previous section, the main `main.typ` file could look like this[1]:

[1]The code below is the main file used to write this document.

Code

```
// main.typ
#import "@local/community-cnam-thesis:0.1.4": *

#let supervisor = ...
#let co-supervisor = ...
#let committee = ...

#show: community-cnam-thesis.with(
    title: [User guide for the Typst thesis template for Cnam theses],
    author: "Mathieu Aucejo",
    thesis-info: (
        doctoral-school: "Sciences des Métiers de l'Ingénieur",
        supervisor: supervisor,
        laboratory: "Laboratoire de Mécanique des Structures et des Systèmes
Couplés",
        co-supervisor: co-supervisor,
        defense-date: "15 juin 2024",
        discipline: "Sciences de l'ingénieur",
        speciality: "Mécanique",
        committee: committee,
        dedication: [Software and cathedrals are almost the same thing: first we
build them, then we pray. \ Sam Redwine],
    ),
    lang: "en",
    open-right: true
)

#show: front-matter

#include "preamble/summary.typ"

#show: main-matter

#tableofcontents
#listoffigures
#listoftables

#part[User guide]

#include "chapters/chapter-main.typ"

#bibliography("bibliography/biblio.bib")

#show: appendix

#part[Appendices]
#include "appendices/appendix-main.typ"

#backcover(resume: ..., abstract: ...)
```

The main file presents the general structure of the document. You can note that the document contains:

- Front matter `#front-matter`, main matter `#main-matter`, and appendix `#appendix` environments.
- A table of contents `#tableofcontents`, a list of figures `#listoffigures`, and a list of tables `#listoftables`.
- Parts `#part`.
- Chapters and appendices imported from other files using the `#include` command.
- A bibliography inserted with the `#bibliography()` command.
- A back cover `#backcover` containing the summary and abstract of the document.

All these elements are detailed in the following chapters.

Chapter

2

Document Formatting

This chapter presents the basic elements for formatting a Typst document. We will particularly discuss title hierarchy, referencing elements, and various content formatting elements.

Content

2.1. Title Hierarchy 26
2.2. Cross-References 27
2.3. Text Formatting 27
2.4. Parts and Page Layout Environments 29
2.5. Back Cover 29

2.1. Title Hierarchy

The `community-cnam-thesis` template supports different levels of titles.

Code

```
= Chapter title (level 1)

== Section title (level 2)

=== Subsection title (level 3)

==== Subsubsection title (level 4)
```

Warning

A four-level numbered outline is generally the maximum acceptable in a thesis. If you need a deeper outline, it is recommended to review the structure of your manuscript.

2.1.1. Numbered and Unnumbered Chapters

By default, chapters are numbered. If you want to create unnumbered chapters (abstract, acknowledgments, introduction, etc.), you can use the following command at the beginning of the corresponding file. For example, for the introduction chapter:

Code

```
#import "@local/community-cnam-thesis:0.1.4": *

#show: chapter-nonum

= Chapter title
```

2.1.2. Numbered and Unnumbered Sections

By default, all titles at levels 1 to 4 are numbered. To insert unnumbered sections or subsections, you must attach the `label` `<nonum-sec>` at the end of the section or subsection title in question. For example:

Code

```
== Unnumbered section title <nonum-sec>
=== Unnumbered subsection title <nonum-sec>
```

2.2. Cross-References

To create cross-references to sections, figures, tables, equations, etc., you must first define a label for the element you want to reference. This is done by adding the `label` `<my-label>` at the end of the element in question:

Code

```
= Chapter title <ch:chapter>

== Section title <s:section>
```

To cite referenced elements, use the command `@my-label`.

typst

```
For more details, see chapter
@ch:structure and section @s:structure-
titles.
```

Rendering

For more details, see chapter 2 and section 2.1.

2.3. Text Formatting

This section presents various text formatting elements.

2.3.1. Emphasis and Code

typ

```
*bold*, _italic_, _*bold and italic*_,
`inline code`, #strike[strikethrough],
#super[superscript], #sub[subscript]
```

Rendering

bold, *italic*, ***bold and italic***, `inline code`, ~~strikethrough~~, superscript, subscript

2.3.2. Hyperlinks

To create hyperlinks, you must use the `#link()` command in Typst (see Typst Documentation for more details).

typst

```
#link("https://www.cnam.fr", "The Cnam
website")

https://www.cnam.fr

#link("mailto:contact@cnam.fr")
```

Rendering

The Cnam website

https://www.cnam.fr

contact@cnam.fr

2.3.3. Lists

There are several types of lists in Typst: bulleted lists, numbered lists, and task lists.

typst

```
- First item
- Second item
  - Subitem 1 (1 tab)
```

Rendering

- First item

```
+ First item
+ Second item
  + First item

#sym.dots.v
5. Fifth item
+ Sixth item

- [x] Task 1
- [ ] Task 2
```

- Second item
  - Subitem 1 (1 tab)

1. First item
2. Second item
   a. First item

   ⋮
5. Fifth item
6. Sixth item

- [x] Task 1
- [ ] Task 2

⚠ Warning

Task lists are suitable for "guide" or "manual" type documents, but are not suitable for a thesis manuscript. It is recommended to use bulleted or numbered lists for thesis writing. This is more neutral and conforms to academic standards.

2.3.4. Quotes

Quotes are used to reference works or ideas developed by other authors.

typst
```
#lorem(5)#footnote[Text of the footnote.]
```

Rendering

Lorem ipsum dolor sit amet.[1]

2.3.5. Footnotes

typst
```
 The Cnam was founded on October 10,
1794#footnote[Vendémiaire 19 of the year
III of the Revolution.].
```

Rendering

The Cnam was founded on October 10, 1794[2].

2.3.6. Page Breaks

To insert a page break, you must use the `#pagebreak()` command in Typst (see Typst Documentation for more details).

</> Code

```
End of content for this section.

#pagebreak()

Beginning of content for the next section.
```

[1] Text of the footnote.
[2] Vendémiaire 19 of the year III of the Revolution.

Warning

It is recommended to use page breaks sparingly, so as not to disturb the reading of the document. It is better to let the layout engine manage page breaks automatically. Prefer to adjust them at the very end of writing, once the content is fixed.

2.4. Parts and Page Layout Environments

Parts are sections at a level higher than chapters. They allow you to group multiple chapters under the same theme. To create a part, you must use the `#part()` command.

Code

```
// Creating a part
#part[Part title]
```

Regarding page layout environments, `community-cnam-thesis` relies on the `bookly` template to structure the document. It is possible to use the `#front-matter`, `#main-matter`, and `#appendix` environments to structure the document in three parts: the front matter, the main content, and the appendices.

To activate these environments, use the following commands at the desired location in the document:

Code

```
#show: front-matter

// Front matter content

#show: main-matter

// Main content

#show: appendix

// Appendices content
```

2.5. Back Cover

The back cover is a page that presents the abstract and the English summary of the document. It is generally used to give readers an overview of the document's content.

Code

```
#backcover(resume: ..., abstract: ...)
```

The `#backcover()` function accepts the following arguments:

Argument

`⟨resume⟩: none` `string` | `content`

French summary of the document

Argument

`⟨abstract⟩: none` `string` | `content`

English abstract of the document

Chapter

3

Figures and tables

This chapter presents the various commands and environments available for inserting figures and tables into the document.

Content

3.1. Figures 32
3.2. Tables 33
3.3. Long and short titles 35
3.4. Creating plots directly in Typst 35

3.1. Figures

The basic syntax for inserting an image is as follows[1]:

Code

```
#image("chemin/vers/image.png", width: 10cm)
```

typst

```
#image("cnam.png")
```

Rendering



Note

Typst supports a number of image formats. The currently supported formats are: PNG, JPEG, GIF, SVG, PDF, WEBP, and Raw Pixel Data.

However, when writing a scientific text, it is often necessary to add a caption to the image and reference it in the text. For this, it is recommended to use Typst's `#figure()` environment. The syntax is as follows[2]:

Code

```
#figure(
  image("chemin/vers/image.png", width: 10cm),
  caption: "Image caption"
) <fig:mon-label>
```

This command inserts an image with a caption and a label for cross-referencing. The label is defined at the end of the `#figure()` environment with the syntax `<mon-label>`. To cite this figure in the text, simply use the command `@mon-label`.

typst

```
#figure(
  image("./images/cnam.png"),
  caption: "Official Cnam logo"
) <fig:logo-cnam>

Figure @fig:logo-cnam shows the official
Cnam logo.
```

Rendering



Figure 3.1 – Official Cnam logo

Figure 3.1 shows the official Cnam logo.

[1] For more information about the `#image()` command, see the Typst documentation

[2] For more information about the `#figure()` environment, see the Typst documentation

You can also insert multiple images into a single figure by using the `#subfigure()` environment from the `bookly` template on which this template is based. The syntax is as follows:

Code

```
#subfigure(
  figure(image("../images/image1.png"), caption: []),
  figure(image("../images/image2.png"), caption: []), <fig:b>,
  columns: (1fr, 1fr),
  caption: [(a) Left image and (b) Right image],
  label: <fig:subfig>,
)
```

typst

```
#subfigure(
  figure(image("./images/cnam.png"), caption: []),
  figure(image("./images/victoire.svg", width: 50%), caption: []), <fig:b>,
  columns: (1fr, 1fr),
  caption: [(a) Cnam logo and (b) Victory stamp],
  label: <fig:subfig>,
)

Figure @fig:subfig shows the official Cnam logo and the allegory of Winged Victory,
associated with the Latin motto "_Docet Omnes Ubique_" (see Figure @fig:b).
```

Rendering

le cnam



(a) (b)

Figure 3.2 – (a) Cnam logo and (b) Victory stamp

Figure 3.2 shows the official Cnam logo and the allegory of Winged Victory, associated with the Latin motto "*Docet Omnes Ubique*" (see Figure 3.2b).

3.2. Tables

Tables can be inserted into the document using Typst's `#table()` environment. The syntax is as follows[3]:

[3]For more information about the `#table()` environment, see the Typst documentation

Code

```
#table(
  columns: 3,
  align: horizon,
  [*Column 1*], [*Column 2*], [*Column 3*],
  [Row 1, Cell 1], [Row 1, Cell 2], [Row 1, Cell 3],
  [Row 2, Cell 1], [Row 2, Cell 2], [Row 2, Cell 3],
)
```

typst

```
#table(
    columns: (1fr,)*2,
    align: center + horizon,
    inset: 5pt,
    [*Shape*], [*Perimeter*],
    [Square with side $a$],[$4a$],
    [Circle with radius $r$],[$2pi r$],
  )
```

Rendering

Shape	Perimeter
Square with side a	$4a$
Circle with radius r	$2\pi r$

As with figures, it is possible to add a caption and a label to a table by using the `#table()` environment from the `bookly` template. The syntax is as follows:

Code

```
#figure(
  table(
    columns: 3,
    align: horizon,
    [*Column 1*], [*Column 2*], [*Column 3*],
    [Row 1, Cell 1], [Row 1, Cell 2], [Row 1, Cell 3],
    [Row 2, Cell 1], [Row 2, Cell 2], [Row 2, Cell 3],
  ),
  caption: "Table caption",
)
```

typ

```
#let mon-tableau = table(
    columns: (1fr,)*2,
    align: center + horizon,
    inset: 5pt,
    [*Shape*], [*Perimeter*],
    [Square with side $a$],[$4a$],
    [Circle with radius $r$],[$2pi r$],
  )

#figure(
  mon-tableau,
  caption: "Table of the perimeters of a
few geometric shapes",
) <tab:perimetres>
```

Rendering

Table 3.1 – Table of the perimeters of a few geometric shapes

Shape	Perimeter
Square with side a	$4a$
Circle with radius r	$2\pi r$

Table 3.1 presents the formulas used to calculate the perimeters of a few geometric shapes.

```
Table @tab:perimetres presents the
formulas used to calculate the perimeters
of a few geometric shapes.
```

Note

Careful readers will have noticed that, to avoid cluttering the code, we defined the table in a variable named `mon-tableau` before inserting it into the `#figure()` environment. This makes it possible to reuse the same table in different parts of the document if necessary and makes the code more readable.

3.3. Long and short titles

The `community-cnam-thesis` template lets you define a long title and a short title for headings or for figures and tables via the `#short-or-long()` function. If applied to headings, the long title is used in the heading itself, while the short title is used in the header. If applied to figures or tables, the long title is used in the figure or table caption, while the short title is used in the list of figures or tables.

typst
```
#figure(
    rect(),
    caption: short-or-long[Representation
of a rectangle][This is a figure
representing a rectangle (see the list of
figures to view the short caption)],
  )
```

Rendering



Figure 3.3 – This is a figure representing a rectangle (see the list of figures to view the short caption)

3.4. Creating plots directly in Typst

Since Typst is a Turing-complete language, it is possible to create plots directly in the document by using, for example, the `lilaq` package. This makes it possible to create charts and visualizations directly in the document without importing external images, while keeping a layout consistent with the rest of the document.

The example below shows how to create a figure with two plots representing the free response of a one-degree-of-freedom mechanical system in the undamped and underdamped cases. For more information about the `lilaq` package, see the lilaq documentation.

typ
```
#import "@preview/lilaq:0.6.0" as lq

// Function computing the free response of a one-degree-of-freedom mechanical system
#let free_response(om0, xi, x0, v0, t) = {
  if xi < 1. {
    // Underdamped
    let Om0 = om0 * calc.sqrt(1 - xi * xi)
    let a = x0
```

```
    let b = (v0 + xi * om0 * x0) / Om0
    calc.exp(-xi * om0 * t) * (a * calc.cos(Om0 * t) + b * calc.sin(Om0 * t))
  } else if xi ≤ 1. {
    // Critically damped
    (x0 + (v0 + om0 * x0) * t) * calc.exp(-om0 * t)
  } else {
    // Overdamped
    let s = om0*calc.sqrt(xi * xi - 1)
    let a = x0
    let b = (v0 + xi * om0 * x0)/s
    calc.exp(-xi * om0 * t) * (a * calc.cosh(s * t) + b * calc.sinh(s * t))
  }
}

#let om0 = 2 * calc.pi * 2            // Natural angular frequency of the system (rad/
s)
#let x0 = 0.02                        // Initial displacement (m)
#let v0 = -0.5                        // Initial velocity (m/s)
#let t = lq.linspace(0, 5, num: 500)  // Time vector (s)

// Create the undamped free-response plot
#let undamped = lq.diagram(
  width: 100%,
  title: [*Undamped free response*],
  xlabel: [Time (s)],
  ylabel: [Displacement (m)],
  margin: 0%,
  lq.plot(t, t => free_response(om0, 0, x0, v0, t),
  color: cnam-colors.primary,
  mark: none,
  ),
)

// Create the underdamped free-response plot
#let underdamped = lq.diagram(
  width: 100%,
  title: [Underdamped free response],
  xlabel: [Time (s)],
  margin: 0%,
  lq.plot(t, t => free_response(om0, 0.1, x0, v0, t),
  color: cnam-colors.primary,
  mark: none,
  ),
)

// Create the grid to display the two plots side by side
#let fig-grid = grid(
  columns: (1fr,)*2,
  column-gutter: 1em,
  [#undamped], [#underdamped],
)

// Create the figure
#figure(
  fig-grid,
  caption: [Free response of a one-degree-of-freedom mechanical system],
) <fig:free-response>
```

```
Figure @fig:free-response shows the free response of a one-degree-of-freedom
mechanical system in the undamped and underdamped cases.
```

Rendering

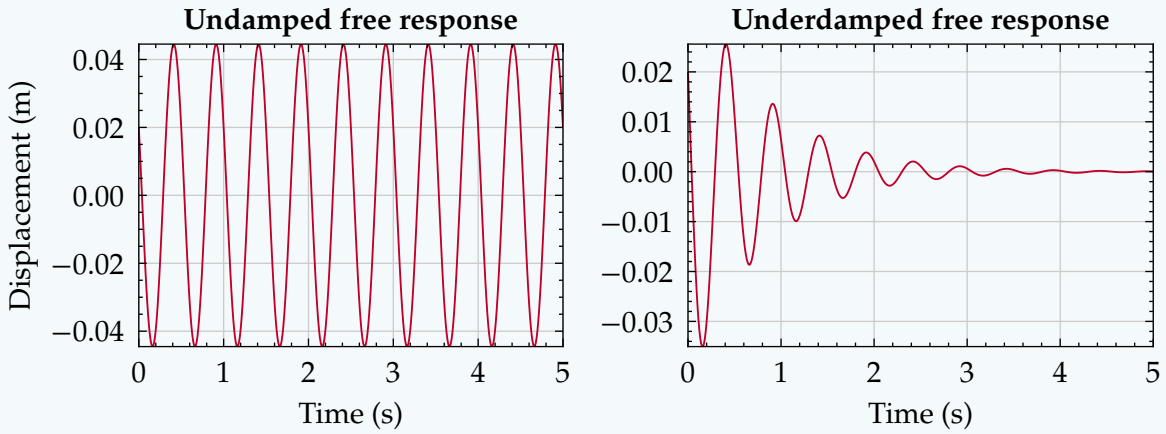


Figure 3.4 – Free response of a one-degree-of-freedom mechanical system

Figure 3.4 shows the free response of a one-degree-of-freedom mechanical system in the undamped and underdamped cases.

Chapter

4

Equations and algorithms

This chapter presents the different ways to write equations and insert algorithms in a thesis document.

Content

4.1. Inline equations 40
4.2. Numbered and unnumbered equations 40
4.3. Advanced environments 41
4.4. Algorithms 42

⚠ Warning

Typst equation syntax differs from LaTeX syntax. For more information about Typst equation syntax, please refer to the official Typst documentation.

To access the full list of mathematical symbols available in Typst, click here.

For the full list of emojis (and yes, it is possible 😈), click there. However, their use should be avoided in an academic document (but I thought it was amusing to mention).

4.1. Inline equations

Inline equations are written between the `$` … `$` symbols and are integrated into the text.

typ
```
#let st = sym.space.thin

The Fourier transform of a function
$f(t)$ is defined by the integral $hat(f)
(xi) = integral_(-oo)^(+oo) f(t) st e^(-2
pi i xi t) dif t$
```

Rendering

The Fourier transform of a function $f(t)$ is defined by the integral $\hat{f}(\xi) = \int_{-\infty}^{+\infty} f(t)\, e^{-2\pi i \xi t}\, \mathrm{d}t$

4.2. Numbered and unnumbered equations

Numbered equations are written between the `$ …  $` symbols and are centered on the page. As with figures and tables, you can add a `label` to refer to them in the text.

⚠ Warning

The difference between numbered equations and inline equations is subtle. Indeed, an inline equation is written as:

`$y = f(x)$`,

whereas a numbered equation is written as (note the space before and after the `$` symbol):

`$ y = f(x) $`.

typ
```
$
i planck (partial Psi)/ (partial t) =
hat(H) space.thin Psi.
$ <eq:schrodinger>

The Schrödinger equation @eq:schrodinger
describes the temporal evolution
#sym.dots
```

Rendering

$$i\hbar\frac{\partial \Psi}{\partial t} = \hat{H}\,\Psi. \tag{4.1}$$

The Schrödinger equation (4.1) describes the temporal evolution …

To avoid numbering an equation, simply add the `label` `<nonum-eq>` after the closing `$` symbol of the equation. For example, the following equation will not be numbered:

```typ
$
i planck (partial Psi)/ (partial t) =
hat(H) space.thin Psi.
$ <nonum-eq>
```

Rendering

$$i\hbar\frac{\partial\Psi}{\partial t} = \hat{H}\,\Psi.$$

4.3. Advanced environments

Equations can also be written in more complex environments, such as systems of equations. For example, to write equations over multiple lines, you can use:

```typ
#let bm(x) = $upright(bold(#x))$
$
  nabla dot.c bm(E) &= rho/epsilon_0, \
  nabla dot.c bm(B) &= 0, \
  nabla times bm(E) &= - (partial bm(B))/
(partial t), \
  nabla times bm(B) &= mu_0 space.thin
bm(J) + mu_0 epsilon_0 (partial bm(E))/
(partial t).
$
```

Rendering

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0}, \tag{4.2a}$$
$$\nabla \cdot \mathbf{B} = 0, \tag{4.2b}$$
$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t}, \tag{4.2c}$$
$$\nabla \times \mathbf{B} = \mu_0\,\mathbf{J} + \mu_0\varepsilon_0\frac{\partial \mathbf{E}}{\partial t}. \tag{4.2d}$$

You can also write the previous equations as a single numbered equation:

```typ
#let bm(x) = $upright(bold(#x))$
$
  nabla dot.c bm(E) &= rho/epsilon_0, \
  nabla dot.c bm(B) &= 0, \
  nabla times bm(E) &= - (partial bm(B))/
(partial t), \
  nabla times bm(B) &= mu_0 space.thin
bm(J) + mu_0 epsilon_0 (partial bm(E))/
(partial t).
$ <equate:revoke>
```

Rendering

$$\begin{aligned}\nabla \cdot \mathbf{E} &= \frac{\rho}{\varepsilon_0},\\ \nabla \cdot \mathbf{B} &= 0,\\ \nabla \times \mathbf{E} &= -\frac{\partial \mathbf{B}}{\partial t},\\ \nabla \times \mathbf{B} &= \mu_0\,\mathbf{J} + \mu_0\varepsilon_0\frac{\partial \mathbf{E}}{\partial t}.\end{aligned} \tag{4.3}$$

Note

Sub-equation numbering is handled by the `equate` package. The `label` `<equate:revoke>` makes it possible to use Typst's native numbering system.

You can also write equations as a system of equations, as follows:

```typ
$
  cases(
    dot(x)(t) &= A x(t) + B u(t),
    y(t) &= C x(t) + D u(t),
  )
$
```

Rendering

$$\begin{cases}\dot{x}(t) = Ax(t) + Bu(t)\\ y(t) = Cx(t) + Du(t)\end{cases} \tag{4.4}$$

The `bookly` template allows the creation of boxed equations using the `#boxeq()` command.

typ
```
$
  #boxeq($y = f(x)$)
$
```

Rendering

$$\boxed{y = f(x)} \tag{4.5}$$

Finally, it is possible to add color to equations:

typ
```
#let colred(x) = text(fill: cnam-
colors.primary, $#x$)
#let colblue(x) = text(fill: blue, $#x$)

$
  colred(y) = colblue(f)(x)
$
```

Rendering

$$y = f(x) \tag{4.6}$$

4.4. Algorithms

Algorithms can be inserted into the thesis document using the `#algorithm()` command, which is based on the package `lovelace`. This command accepts the following parameters:

Argument

⟨caption⟩: none — `string` | `content`

Algorithm caption. If `none`, no caption is displayed.

Argument

⟨line-numbering⟩: "1" — `string`

Line numbering for the algorithm.

If `none`, no line numbering is displayed.

typ
```
#algorithm(caption: [My algorithm])[
  + do something
  + *while* still something to do
    + do even more
    + *if* not done yet *then*
      + wait a bit
      + resume working
    + *else*
      + go home
    + *end*
  + *end*
] <alg:example>

#noindent Algorithm @alg:example is an
example of a simple, completely useless
algorithm.
]
```

Rendering

Algorithm 4.1 – My algorithm

```
1  do something
2  while still something to
   do
3  | do even more
4  | if not done yet then
5  | | wait a bit
6  | | resume working
7  | else
8  | | go home
9  | end
10 end
```

Algorithm 4.1 is an example of a simple, completely useless algorithm.

Chapter

5

Callout boxes

Callout boxes are formatting elements used to highlight specific information in a document. They are often used to draw the reader's attention to important points, tips, warnings, or additional notes.

Content

5.1. Available boxes 44
5.2. Special cases for code boxes 45

5.1. Available boxes

The `community-cnam-thesis` template provides several types of informational boxes, each with a specific style and purpose. Here are the main available boxes:

- `#info-box`: Used to provide general information or useful advice.

```typ
#info-box[This is an information box.]
```

Rendering

Note

This is an information box.

- `#tip-box`: Used to provide tips or recommendations.

```typ
#tip-box[This is a tip box.]
```

Rendering

Tip

This is a tip box.

- `#warning-box`: Used to warn the reader about a potential danger or important information.

```typ
#warning-box[This is a warning box.]
```

Rendering

Warning

This is a warning box.

- `#important-box`: Used to emphasize crucial information or key points.

```typ
#important-box[This is an important box.]
```

Rendering

Important

This is an important box.

- `#proof-box`: Used to present proofs or mathematical demonstrations.

```typ
#proof-box[This is a proof box.]
```

Rendering

Proof

This is a proof box.

- `#question-box`: Used to ask questions or propose problems to solve.

```typ
#question-box[This is a question box.]
```

Rendering

Question

This is a question box.

- `#code-box`: Used to present source code or code snippets.

```typ
#code-box[This is a code box.]
```

Rendering

Code

This is a code box.

Warning

Informational boxes should be used carefully and sparingly to avoid overloading the document.

5.2. Special cases for code boxes

To fill a code box, you can use Typst's native syntax:

````typ
#code-box[```julia
import LinearAlgebra

A = rand(3, 3)
b = rand(3)

x = A \ b
```]
````

Rendering

Code

```julia
import LinearAlgebra

A = rand(3, 3)
b = rand(3)
```
```

```
x = A \ b
```

You can add line numbering by using dedicated packages, such as `zebraw`:

typ

```
#code-box[#zebraw(
  lang: false,
  ```julia
  import LinearAlgebra

  A = rand(3, 3)
  b = rand(3)

  x = A \ b
  ```
)]
```

Rendering

Code

```
1 import LinearAlgebra
2
3 A = rand(3, 3)
4 b = rand(3)
5
6 x = A \ b
```

Note

The `zebraw` package is an explicit dependency of the `community-cnam-thesis` template. It can therefore be used directly without importing it explicitly.

To go further, you can consult the documentation for `zebraw`. Note that other packages exist for code formatting, such as `codly` or `codelst`. However, these packages are not dependencies of the `community-cnam-thesis` template and must therefore be explicitly imported if you want to use them.

Chapter

6

References

This chapter presents the bibliographic citation system used in the Cnam thesis template. It is based on the IEEEtran citation style, adapted for French. The style file used is `IEEEtran-francais.csl`, which is included in the `community-cnam-thesis` template. The glossary and acronym system is also presented.

Content

6.1. Bibliographic references 48
6.2. Glossary, acronyms, and nomenclatures 49

6.1. Bibliographic references

This section explains how to insert citations into your document.

6.1.1. File formats

Typst natively supports bibliography files in BibTeX format. However, Typst introduces a new bibliography file format called Hayagriva, which is a `yaml` file containing bibliographic references. For more information about the Hayagriva format, please refer to its documentation.

To insert a bibliography, use the `#bibliography()` command and specify the path to the bibliography file.

Code

```
// Pour un fichier BibTeX
#bibliography("bibliographie/biblio.bib")

// Pour un fichier Hayagriva
#bibliography("bibliographie/biblio.yaml")
```

For more information about the `#bibliography()` command, please refer to the official Typst documentation.

Tip

Zotero (with the Better BibTeX connector) or JabRef can export your bibliography directly in BibTeX format. Note that JabRef can also export your bibliography in Hayagriva format.

6.1.2. Basic syntax

Citations are inserted into the text either by using the `#cite()` command, or by using the same syntax as cross-references `@key`.

typ

```
Scientific typesetting was revolutionized
by Donald Knuth in the 1970s @Knu84.
```

Rendering

Scientific typesetting was revolutionized by Donald Knuth in the 1970s [1].

6.1.3. Multiple references

To insert multiple references in the text, simply chain them.

typ

```
Typst is a new markup language developed
by two German students, Laurenz Mädje and
Martin Haug, as part of their master's
thesis in computer science @Mad22 @Hau22.
```

Rendering

Typst is a new markup language developed by two German students, Laurenz Mädje and Martin Haug, as part of their master's thesis in computer science [2, 3].

6.2. Glossary, acronyms, and nomenclatures

Several Typst packages exist to create glossaries and acronyms. The `community-cnam-thesis` template does not include specific packages for creating glossaries and acronyms. However, you can use the following packages:

- **Glossaries**
  - Glossy
  - Glossarium
  - Gloss-awe
- **Acronyms**
- Acrostiche
- Acrotastic
- Abbr
- I-am-acro
- **Nomenclatures**
- Nomos

Chapter

7

Review comments

This section presents the different review and annotation features available in the `community-cnam-thesis` template. These features allow authors and reviewers to collaborate efficiently by adding comments, margin notes, and annotations directly in the document.

Content

7.1. Collaborative comments 52
7.2. Comment types 52
7.3. Annotation syntax 53
7.4. Table of annotations 55

7.1. Collaborative comments

To enable comments, simply insert the following command in the relevant file:

Code

```
#show: activate-comment
```

To disable comments, simply insert the following command in the relevant file:

Code

```
#show: deactivate-comment
```

Enabling comments creates a margin area on the right side of the document, where comments can be displayed.

Note

The `activate-comment` is a shortcut for `marginalia.setup.with( ... )`, and the `deactivate-comment` command is a shortcut for `page.with( ... )`. If the activation command is defined in an included file, it is not necessary to disable comments in subsequent included files.

7.2. Comment types

The `community-cnam-thesis` template provides different comment types, each with a specific icon to distinguish them visually:

- **`note`** : Used to add informational notes or additional explanations.
- **`comment`** : Used to add general comments or suggestions.
- **`question`** : Used to ask questions or request clarification.
- **`todo`** : Used to indicate tasks to complete or items to check.

7.3. Annotation syntax

Note

The comment environment has been enabled for this section.

Annotations can be added using the `#comment()` command, whose arguments are as follows:

Argument

`(by): "Reviewer"` — `string` | `content`

Comment author

Argument

`(type): "note"` — `string`

Comment type

Argument

`(inline): false` — `bool`

If `true`, the comment is inserted inline. If `false`, it is inserted in the margin.

Argument

`(color): blue` — `string` | `color`

Annotation box color

Argument

`(icon): true` — `bool`

Display the comment icon for inline comments. If `true`, the comment icon is displayed in the margin. If `false`, it is not displayed.

Argument

`(..args)` — `dictionary`

Additional arguments for annotation customization.

Note

The `#comment()` command is built from the `#note()` command provided by the **`marginalia`** package for margin notes, while it uses the built-in `#block()` command for inline notes. Therefore, `#comment()` inherits the parameters of these two commands. For more information about available parameters, please refer to the `marginalia` package documentation.

Warning

Due to the current implementation of the `#comment()` command, some parameters of the `#inline-note()` command are not yet supported. This is notably the case for the **`par-break`** parameter.

To add a margin comment of type `note` with a blue annotation box, use the following command:

Code

```
#comment(by: "Abbé Grégoire", color: blue)[This is a blue comment.]
```

As you can see, the previous command does create a margin comment of type `note` with a blue annotation box. [1]. An attentive reader will notice that the inserted annotation is marked in the text by its icon, color, and number.

1
Abbé Grégoire : This is a blue comment.

You can go further by creating annotation boxes associated with a specific reviewer. This makes it possible to distinguish each reviewer's comments with a specific color. For example, let's create two reviewers, Abbé Grégoire and Henri Tresca (who are the advisors of this fictional thesis).

Code

```
#let ab-comment = comment.with(by: "Abbé Grégoire", color: blue)
#let ht-comment = comment.with(by: "Henri Tresca", color: cnam-
color.primary)
```

By doing this, you can add comments from each reviewer using the `ab-comment` and `ht-comment` commands [2] [3]. These notes were created with the following commands:

2
Henri Tresca : I think this is great!

3
Abbé Grégoire : I am not sure I understand this remark, Henri?

Code

```
By doing this, you can add comments from each reviewer using the
`ab-comment` and `ht-comment` commands#ht-comment(type: "comment",
dy: -3.5em)[I think this is great!]#ab-comment(type: "question")[I
am not sure I understand this remark, Henri?].
```

Comments inserted in the margins should generally be relatively short so as not to overload the layout. However, it is possible to add longer comments using the `inline: true` argument. This inserts the comment directly into the text, which is particularly useful for detailed explanations or longer discussions. [4]

4
Abbé Grégoire : Since I don't have any ideas for writing a long and funny text, I will use a classic `#lorem()`. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

It is also possible to highlight a passage of text and associate a comment with it. To do this, simply use the `#highlight-comment()` command, which highlights the text and adds an associated comment. The command is as follows:

Code

```
#highlight-comment(by: "Reviewer A", type: "comment", color: green,
highlight-body: [Highlighted text])[Associated comment]
```

In the next section, we will present how to create the annotation list. [5]

5 **Mathieu Aucejo :** I wonder whether this is really necessary?

Note

The `#highlight-comment()` command is built from the `#comment()` command. It inherits this command's parameters except for the `inline` parameter.

7.4. Table of annotations

To make reading and navigating comments easier, the `community-cnam-thesis` template also provides an annotation table. This table lists all comments added to the document, with their number, author, and type. To insert it in the document, simply use the `#listofnotes` command at the desired location.

Code

```
#listofnotes
```

Conclusion

This user guide for the `community-cnam-thesis` Typst template was written to help students and researchers prepare theses that comply with Cnam requirements. It provides practical instructions for configuration, structure, bibliography management, and advanced features such as annotations.

Feature summary

Feature	Syntax	Chapter
Template import	`#import "@local/community-cnam-thesis:0.1.4": *`	Chapter 1
Template initialization	`#show: community-cnam-thesis.with( ... )`	
Heading hierarchy	`= Chapter`, `== Section`, `=== Subsection`	Chapter 2
Unnumbered chapters	`#show: chapter-nonum`	
Unnumbered sections	`== Section <nonum-sec>`	
Cross-references	`<my-label>` + `@my-label`	
Layout environments	`front-matter`, `main-matter`, `appendix`	
Parts	`#part[Part title]`	
Back cover	`backcover( ... )`	
Insert image	`image("path/to/image.png")`	Chapter 3
Insert table	`table( ... )`	
Numbered figures/tables	`figure( ... )`	
Subfigures	`subfigure( ... )`	
Inline equations	`$E=mc^2$`	Chapter 4
Numbered equations	`$ E=mc^2 $`	
Unnumbered equations	`$ E=mc^2 $ <nonum-eq>`	
Info box	`#info-box[ ... ]`	Chapter 5
Tip box	`#tip-box[ ... ]`	
Warning box	`#warning-box[ ... ]`	
Important box	`#important-box[ ... ]`	
Proof box	`#proof-box[ ... ]`	
Question box	`#question-box[ ... ]`	
Code box	`#code-box[ ... ]`	

Feature	Syntax	Chapter
Insert bibliography	`#bibliography("path/to/biblio.bib")`	
Citation in text	`@key`	Chapter 6
Multiple references	`@key1 @key2`	
Enable comments	`#show: activate-comment`	
Comment	`#comment( ... )[Comment text]`	Chapter 7
Highlighted comment	`#highlight-comment( ... )[text][comment]`	

Bibliography

[1] D. E. Knuth, *The TeXbook*, vol. A. in Computers & Typesetting, vol. A. Reading, Massachusetts: Addison-Wesley, 1984.

[2] L. Mädje, "Typst: A programmable markup language for typesetting," Master's thesis, 2022.

[3] M. Haug, "Fast typsetting with incremental compilation," Master's thesis, 2022.

Appendices

Appendix

A

PDF/A-1b validation

This annex provides instructions for validating that the generated PDF document complies with the PDF/A-1b standard.

Content

A.1. Submission on theses.fr 64
A.2. CINES validation 64
A.3. Typst compilation to PDF/A-1b 64

A.1. Submission on theses.fr

The theses.fr platform allows you to submit theses and research dissertations. For the document to be accepted, it must comply with the PDF/A-1b standard (ISO 19005-1).

A.2. CINES validation

The online service facile.cines.fr is provided by CINES (Centre Informatique National de l'Enseignement Supérieur) to validate PDF document compliance with the PDF/A-1b standard. It is recommended to use this service to check that your document meets formatting requirements before submitting it to theses.fr.

A.3. Typst compilation to PDF/A-1b

Typst supports several PDF output standards, including PDF/A-1b. To generate a document compliant with this standard, several options are available:

1. Command line

Code

```
typst compile main.typ thesis.pdf --pdf-standard a-1b
```

2. If you use the Typst web app, you can select the PDF/A-1b format in the export options before downloading the document (see Figure A.1).

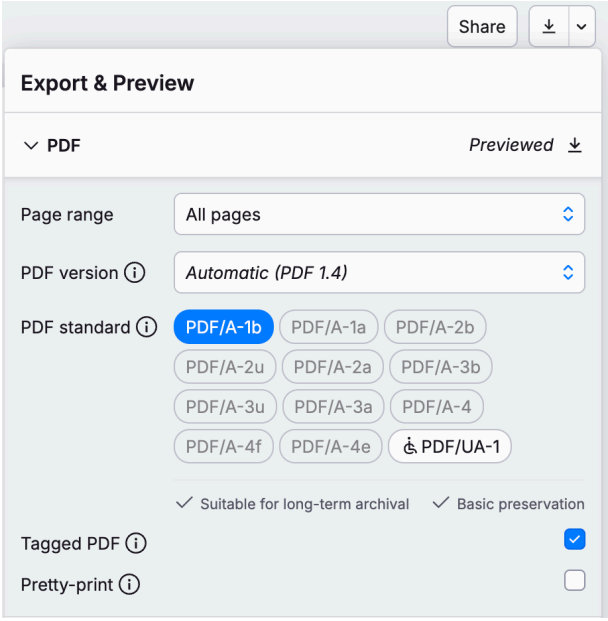


Figure A.1 – Selecting the PDF/A-1b format in the Typst web app

3. If you use the Tinymist extension for Visual Studio Code, you can configure the PDF/A-1b format in the Typst extension settings. To do this, select the extension → Export Tool. Then choose the PDF/A-1b format in the "PDF Validator" dropdown list (see Figure A.2).

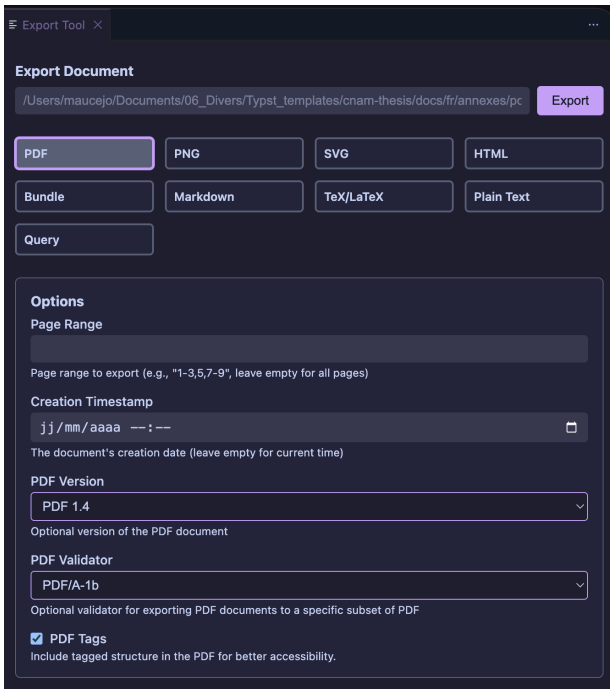


Figure A.2 – Selecting the PDF/A-1b format in the Tinymist extension for Visual Studio Code

⚠ Warning

The PDF/A-1b format is a strict standard that imposes certain constraints on the document's content and structure. For example, this standard does not accept images with transparency. Make sure to check your document after compilation to ensure it complies with this standard.



le cnam

Mathieu Aucejo

Cnam thesis template
User's guide

Résumé : Ce guide a pour objectif de présenter le template de thèse du Cnam, développé avec le langage de mise en page Typst. Il est destiné à aider les doctorants et les chercheurs à rédiger leur thèse conformément aux normes du Cnam, en utilisant un formatage cohérent et professionnel. Le guide couvre l'installation du template, la structure des fichiers, l'insertion de contenu, la gestion des références bibliographiques, et la personnalisation du document.

Abstract: This guide aims to present the Cnam thesis template, developed with the Typst typesetting language. It is intended to help doctoral students and researchers write their thesis in accordance with Cnam standards, using consistent and professional formatting. The guide covers template installation, file structure, content insertion, bibliographic reference management, and document customization.